LLM-JEPA on a Crowd-Trained 0.66B Base Model: A Pre-Registered Study on a Shared Tesla T4

Xavier Callens
Project HiveMind

14 November 2026

Abstract

LLM-JEPA adds a joint-embedding predictive term to the next-token objective of language models and reports large gains on fine-tuning tasks with naturally paired views, including +11.4 points at LoRA rank 64 on NL-RX-SYNTH for an instruction-tuned 1B model. We test whether this transfers to *Crowd-v1*, a 658M-parameter base model trained by volunteers in the CrowdGPT project, under the constraints of a single Tesla T4 shared with live inference services (at most 2.7 GB per experiment, 4.3 GB of VRAM always left free). In a pre-registered pilot (LoRA r64, 3 seeds per arm, 1,000 test examples), LLM-JEPA brings no gain ($16.90 \pm 1.06\%$ vs. $17.07 \pm 0.93\%$ exact match) and degrades general held-out loss by +0.90 nat. The JEPA term collapses to $1 - \cos \leq 5 \times 10^{-4}$. A compute-matched screen of ten pre-registered hypotheses, in the style of *autoresearch*, shows that stop-gradient, centering and InfoNCE remove the collapse and the damage, and that a weak JEPA term ($\lambda = 0.1$ with loss dropout) combined with stop-gradient or centering approaches, but does not beat, plain fine-tuning. An inference-only diagnosis locates Crowd-v1's main weakness elsewhere: its gap to Qwen3-0.6B is +3.84 nat on Python code and +2.44 on mathematics, against +0.36 on its own data. We also find and fix a held-out evaluation bug that ignored the model's engram table (+0.35 nat). Whether the null result is specific to Crowd-v1 remains open; we describe a ready, pre-registered test that separates model from recipe.

1 Introduction

Small language models trained by volunteer compute face a token budget far below that of industrial models. LLM-JEPA [1] proposes a way to extract more from data: keep the autoregressive model and its inference unchanged, but add, during training, a term that asks the final hidden state of one view of a content (e.g. a natural language description) to predict the embedding of another view (e.g. the corresponding regular expression). The paper reports gains on several fine-tuning tasks, with the largest on NL-RX-SYNTH [10].

We study this method for *Crowd-v1*, the 1B-class (658M trunk parameters) model of the CrowdGPT project, trained by volunteer GPUs with periodic delta aggregation, a scheme close to DiLoCo [11]. Our constraints are those of a small open project: a single Tesla T4 that also serves inference, a hard rule to keep 4.3 GB of VRAM free, and no budget for larger accelerators. Within those constraints we ask three questions: (i) what is the real held-out quality of Crowd-v1; (ii) does LLM-JEPA improve it on a paired-view task, and can it be tuned to; (iii) where else could Crowd-v1 be improved. All decisive experiments were pre-registered: protocols and success criteria were written before any result.

Contributions.

- An evaluation fix: the held-out loss check of the project ignored the model's engram table, which costs 0.35 nat (Section 4).

- A pre-registered null result for LLM-JEPA on Crowd-v1 with LoRA r64, with a measured collapse of the JEPA term and a +0.90 nat degradation of general loss (Section 5).
- A compute-matched screen of 10 hypotheses (5 tested) identifying two levers that combine: weak JEPA pressure and an anti-collapse mechanism (Section 6).
- A per-domain diagnosis showing that the data mix, not the training objective, is Crowd-v1's main limitation (Section 7).
- An open, ready-to-run test of whether the null result is model- or recipe-specific (Section 8).

2 Background

LLM-JEPA. Given two views T (text) and C (code) of the same content, LLM-JEPA minimizes

$$\mathcal{L} = \sum_{\ell} \mathcal{L}_{\mathrm{NTP}}(T, C) + \lambda\, d(\mathrm{Pred}(\mathrm{Enc}(T)), \mathrm{Enc}(C)), \qquad d = 1 - \cos, \tag{1}$$

where $\mathrm{Enc}(x)$ is the last-layer hidden state of the last token of x. The predictor reuses the model's own weights through k appended predictor tokens ($k = 0$ is the identity). Following the JEPA line of work [3, 4], the next-token term is what prevents collapse; the paper reports that dropping it yields empty outputs. Training cost is about 2×; inference is unchanged [1]. The reference code applies no stop-gradient.

Anti-collapse mechanisms. Joint-embedding methods avoid trivial solutions with asymmetric stop-gradient [5], contrastive negatives (InfoNCE, [6]) or variance/covariance regularization [7]. Contextual embeddings of language models are also known to be anisotropic [8], which makes a raw cosine between final states easy to drive towards 1.

autoresearch. *autoresearch* [2] runs an agent loop over a fixed evaluation and a fixed wall-clock training budget (5 minutes on an H100), with one change per experiment, kept if the single metric improves and reverted otherwise.

3 Setup

Model. `Vxtzq/Crowd-v1`, revision `5ac3916` (global step 366): 24 layers, width 1536, grouped-query attention (16/4 heads), SwiGLU, tied head, 657,787,392 trunk parameters, plus a bigram *engram* table of 227,865 buckets × 1536 (350,000,640 parameters) added to the token embedding. Tokenizer: Qwen3 (vocabulary 151,669). Reference model: Qwen3-0.6B [12], same tokenizer.

Hardware and safety. One NVIDIA Tesla T4 (15 GB) that also hosts inference services (about 5.1 GB) and other tenants (an ollama model of about 2.4 GB that loads and unloads). Every experiment runs under a guard that:

- keeps free VRAM ≥ 4300 MiB at every step;
- caps the process at 2.7 GB;
- pauses while inference requests are in flight;
- checkpoints and resumes after aborts;
- yields to the owner's training jobs.

Table 1: E0: held-out next-token CE (nat/token), mean (sd) over 12 windows of 2,048 tokens.

Model	chunk 1026 @0	chunk 1026 @200 MB	wikitext-2 test
Crowd-v1 + engram (fp16 = fp32 = bf16)	3.030 (0.355)	3.194 (0.403)	4.855 (0.345)
Crowd-v1 without engram	3.376 (0.330)	3.570 (0.377)	5.055 (0.308)
Qwen3-0.6B	2.536 (0.300)	2.776 (0.293)	3.020 (0.268)

Fine-tuning task. NL-RX-SYNTH [10]: 8,000 training pairs; the test set is the first 1,000 of 2,000 examples; exact string match as in the reference evaluator. Prompt: an instruction, the query and `Regex:`; the answer is terminated by `<|endoftext|>`.

Training. LoRA [9] rank 64 ($\alpha = 64$) on all attention and MLP projections; base and engram frozen in fp16; AdamW, effective batch 16. In the JEPA arm, view 1 is the bare query and view 2 the regex. Both views are packed in one sequence with a block-causal mask, and the engram bigram input is reset at the start of view 2, so that view 2 is exactly a separate pass. The next-token loss comes from the normal prompt+answer pass. The embedding is the final (post-norm) state of each view's last token.

General-loss check. Mean next-token CE on 12 fixed 2,048-token windows of a CrowdGPT chunk not used in fine-tuning. Base model: 3.194 nat.

4 Held-out quality of Crowd-v1 (E0)

Table 1 reports held-out CE on 12 windows per source.

- The model is healthy: about 3 nat on its own distribution, against 11.93 for a uniform guess.
- An earlier reading of 8–11 nat came from an older revision measured by a loss check that omitted the engram table. Omitting it alone costs +0.35 nat.
- fp16, fp32 and bf16 agree to 10^{-3} nat on the T4.
- Cohort training reads 0.78 passes per data sub-slice, and the gap between a recently trained region and a distant one (0.16 nat) is smaller than Qwen's (0.24): there is no sign of memorization.

5 Pre-registered pilot: SFT vs. SFT + LLM-JEPA (P1)

Protocol. Written before any run:

- LoRA r64, lr 2×10^{-4} with cosine schedule, 3 epochs (1,500 steps), seeds 0–2.
- JEPA with $\lambda = 1$, $k = 0$.
- Criteria:
  1. a floor of $\geq 5\%$ for SFT seed 0;
  2. success if $\Delta = \overline{\text{JEPA}} - \overline{\text{SFT}} > 2\,\sigma_{\text{pooled}}$ and the JEPA general loss is $\leq$ SFT +0.05 nat;
  3. a "paper-like" gain if, in addition, $\Delta \geq 10$ points.

Table 2: P1: exact match on 1,000 NL-RX-SYNTH test examples and general held-out loss (base 3.194). LoRA r64, lr 2×10^{-4}, 3 epochs.

Seed	SFT (%)	JEPA λ=1 (%)	Δ	SFT general	JEPA general
0	16.0	15.7	-0.3	3.789	4.605
1	17.7	17.7	0.0	3.719	4.652
2	17.5	17.3	-0.2	3.830	4.770
mean $\pm$ sd	17.07 ± 0.93	16.90 ± 1.06	-0.17	3.779	4.676

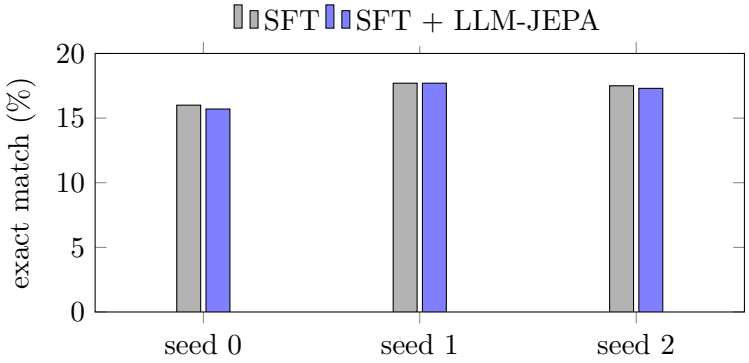


Figure 1: P1 accuracy per paired seed: no gain from LLM-JEPA ($\lambda = 1$, $k = 0$).

Results. Table 2 and Figure 1.

- The floor is met.
- Criteria 2 and 3 are not: $\Delta = -0.17$ points against a threshold of 1.99, with paired differences -0.3, 0.0 and -0.2.
- General loss is 0.90 nat worse with JEPA.
- An exploratory $\lambda = 4$ run is worse on both counts (15.4%, 5.012 nat).
- A secondary prefix-match metric shows no difference either (seeds 1–2: SFT 20.6/19.4, JEPA 19.5/19.8).
- The JEPA term drops from 0.15 at step 25 to $\leq 5 \times 10^{-4}$ in every run.
- JEPA costs 1.45–1.61 s per step against 0.71–0.83 for SFT (about 1.9×).

6 Compute-matched screen of ten hypotheses

Protocol. Adapted from autoresearch [2]:

- a fixed 600 s training budget, a T4 equivalent of 5 minutes on an H100;
- one seed, LoRA r64, lr 2×10^{-4} with a time-based schedule;
- metric: answer-token CE on 500 development examples never trained on;
- guard rail: general loss ≤ 3.446 (SFT +0.05);
- a change is kept only if it beats SFT.

Table 3: autoresearch screen (600 s budget, 1 seed). val CE on 500 dev examples; general loss guard rail ≤ 3.446. Combinations marked † were added after the screen and do not enter the pre-registered selection.

Run	val CE	Δ vs SFT	general	guard	steps	JEPA term (end)
SFT	**2.503**	—	3.396	—	496	—
JEPA λ=1	2.735	+0.231	4.159	×	385	0.0025
H1 λ=0.1 + loss dropout 0.5	2.513	+0.009	3.474	×	545	0.023
H2 stop-gradient	2.803	+0.300	3.399	✓	365	0.081
H3 centered cosine	2.962	+0.459	3.323	✓	355	0.007
H4 InfoNCE (T=0.07)	3.274	+0.770	3.253	✓	305	0.108
H6 layer-16 alignment	2.608	+0.105	4.352	×	482	0.0017
H1+H2†	2.540	+0.037	3.430	✓	546	0.179
H1+H3†	2.505	+0.001	3.452	×	572	0.197

Ten hypotheses were written before the first run, each tied to a P1 observation: weak pressure (H1); stop-gradient on the target (H2); centered cosine (H3); InfoNCE (H4); $k = 1$ (H5); intermediate-layer alignment (H6); mean pooling (H7); LoRA r512 (H8); warm start (H9); VICReg (H10). Five were tested (H1–H4, H6). H8 does not fit the 2.7 GB cap; the others were excluded for cost, overlap or a lower prior.

Results. Table 3.

- No variant beats SFT, so the pre-registered rule keeps none.
- Ranking the guard-passing variants selects H2, H3 and H4: the three that stop the collapse. Each ends at or below SFT's general loss, but slows task learning at equal compute.
- Weak pressure (H1) almost matches SFT on the metric but slightly fails the guard rail.
- Two post-hoc combinations, H1+H2 and H1+H3, confirm that the levers combine. H1+H2 is the best guard-passing variant (+0.037 nat), and H1+H3 ties SFT (+0.001 nat) while missing the guard by 0.006.
- Moving the alignment to layer 16 (H6) does not prevent the collapse.

7 Where Crowd-v1 is weak: per-domain diagnosis (P0)

Figure 2 compares Crowd-v1 with Qwen3-0.6B on 8 windows per domain.

- On CrowdGPT's own data the gap is small (+0.36 nat).
- It is large on Python code (+3.84), mathematics (+2.44), encyclopedic text (+1.80) and science (+1.50).
- Part of each gap is expected, since Qwen3 was trained on far more code and mathematics. But a 3.8 nat gap indicates that Crowd-v1 has seen almost no code.

This points at the data mix as the cheapest and largest lever, independent of the training objective.

8 Discussion

Is the null result specific to Crowd-v1? Not established. The paper's LoRA table on NL-RX-SYNTH (lr 2×10^{-5}, $\lambda = 1$, $k = 1$, 5 runs) reports $21.09 \pm 1.90 \rightarrow 32.46 \pm 1.26$ at rank 64 [1]. LoRA rank therefore does not explain our null: our SFT (17.1%) is close to theirs. The remaining differences are:

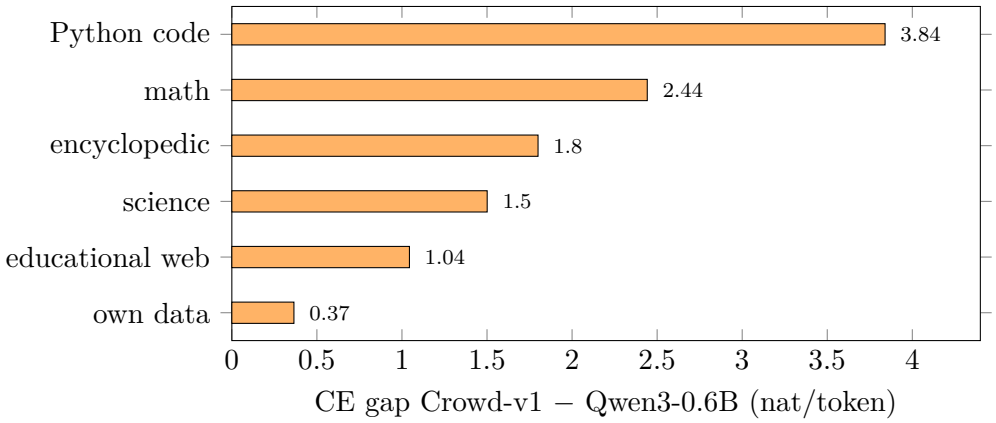


Figure 2: P0: per-domain held-out gap to Qwen3-0.6B (same tokenizer, 8 windows of 2,048 tokens per domain).

- a 10× larger learning rate;
- $k = 0$;
- 3 instead of about 4 epochs;
- the model itself. The paper evaluates instruction-tuned models only; Crowd-v1 is a base model with an engram table.

The collapse we observe (final-state cosine $\to 1$ within a few hundred steps, with general-loss damage) is consistent with the anisotropy of final hidden states [8]. A large learning rate may reach this shortcut faster.

A ready test (P2). The paper's LoRA recipe (lr 2×10^{-5}, $\lambda = 1$, $k = 1$ with a reserved token, 4 epochs) applied to Qwen3-0.6B and to Crowd-v1, with paired seeds. The reading of all four outcomes was fixed in advance:

- a gain on both points to our recipe;
- a gain on Qwen only points to the model;
- no gain on either means the effect does not reproduce at this scale.

P2 is implemented but not yet run: the T4 is reserved for training.

Limitations.

- One task, one adapter rank and one seed in the screen.
- Compute-matched comparisons penalize JEPA's per-step cost by design.
- The screen's loss-dropout arms drew their dropout decision from the data-order RNG, so their batches were not paired with SFT's. This adds noise but no bias, and was fixed afterwards.
- The Qwen3-0.6B checkpoint used as reference is post-trained.
- No French data was measured.

9 Improvement proposals

1. **Data mix.** Add code, mathematics and reference text to the CrowdGPT corpus, after license review by the maintainer. Track the per-domain gaps on every checkpoint with the released evaluation tool.
2. **Evaluation.** Keep the held-out loss check engram-aware, and bind the engram table in the signed receipt.
3. **JEPA, if pursued.** Do not wire it into the volunteer client yet. If used, start from $\lambda = 0.1$ with loss dropout 0.5 and stop-gradient (the best guard-passing variant). Run P2, then LoRA r512 on a 24 GB GPU, where the paper reports its largest LoRA gain.
4. **Other models.** The natural pairs of views for other project models (docstring $\leftrightarrow$ code, question $\leftrightarrow$ answer, statement $\leftrightarrow$ proof) are listed with a gate: nothing is built unless P2 shows a JEPA gain on Qwen3-0.6B.

Reproducibility

Code, protocols written before the runs, and raw results are in the HiveMind repository:

- code and protocols: `scripts/crowdgpt_t4/`;
- raw results: `artifacts/experiments/CROWDGPT_*`;
- reports: `docs/CROWDGPT_LLM_JEPA_RESULTS.md` and `docs/CROWDGPT_LLM_JEPA_STUDY.md`.

The opt-in loss module (`crowd_jepa.py`), the per-domain evaluation tool and the data-mix proposal are proposed to the CrowdGPT repository. Model weights and data are pinned by revision.

Acknowledgments

We thank the CrowdGPT maintainer and its volunteer contributors for the open model and data. Experiment code, runs and the first draft of this report were produced with an AI coding assistant (Claude, Anthropic) under the author's direction. Every number was checked against the stored result files.

References

[1] H. Huang, Y. LeCun, R. Balestriero. LLM-JEPA: Large Language Models Meet Joint Embedding Predictive Architectures. arXiv:2509.14252, 2025.

[2] A. Karpathy. autoresearch. https://github.com/karpathy/autoresearch, 2026.

[3] Y. LeCun. A Path Towards Autonomous Machine Intelligence. OpenReview, 2022.

[4] M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. CVPR, 2023.

[5] J.-B. Grill et al. Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning. NeurIPS, 2020.

[6] A. van den Oord, Y. Li, O. Vinyals. Representation Learning with Contrastive Predictive Coding. arXiv:1807.03748, 2018.

[7] A. Bardes, J. Ponce, Y. LeCun. VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. ICLR, 2022.

[8] K. Ethayarajh. How Contextual are Contextualized Word Representations? EMNLP, 2019.

[9] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.

[10] N. Locascio, K. Narasimhan, E. DeLeon, N. Kushman, R. Barzilay. Neural Generation of Regular Expressions from Natural Language with Minimal Domain Knowledge. EMNLP, 2016. arXiv:1608.03000.

[11] A. Douillard et al. DiLoCo: Distributed Low-Communication Training of Language Models. arXiv:2311.08105, 2023.

[12] Qwen Team. Qwen3 Technical Report. arXiv:2505.09388, 2025.

[13] CrowdGPT. https://github.com/Vxtzq/CrowdGPT; model https://huggingface.co/Vxtzq/Crowd-v1.